

Plan Switch Classification Analysis

1. Query 1 Analysis:

Execution Times and Classification

Plan Switch	Dates ($q_i \rightarrow q_j$)	RT(P_i, q_i)	RT(P_j, q_j)	RT(P_i, q_j)	RT(P_j, q_i)	Classification
S1,2	1992-01-07 $\rightarrow$ 1992-01-08	19.63 ms	11.85 ms	19.38 ms	29.89 ms	CORRECT
S2,3	1992-01-27 $\rightarrow$ 1992-01-28	491.32 ms	392.76 ms	381.54 ms	444.17 ms	DELAYED
S3,4	1992-03-29 $\rightarrow$ 1992-03-30	1402.40 ms	65502.06 ms	14090.69 ms	2154.57 ms	EARLY

Justifications

Switch 1 (1992-01-07 $\rightarrow$ 1992-01-08): CORRECT

The optimizer made the switch at the correct time.

At q_i , the chosen plan P_i was faster than the forced alternative P_j : 19.63 ms < 29.89 ms

At q_j , the newly chosen plan P_j was faster than the forced original plan P_i : 11.85 ms < 19.38 ms

Switch 2 (1992-01-27 $\rightarrow$ 1992-01-28): DELAYED

The optimizer switched to P_j on Jan 28, but P_j was already outperforming P_i on Jan 27.

We see that RT(P_j, q_i) is 444.17 ms, which is faster than the optimizer's chosen RT(P_i, q_i) of 491.32 ms.

Since P_j was already faster, the switch should have occurred earlier.

Switch 3 (1992-03-29 $\rightarrow$ 1992-03-30): EARLY

The optimizer switched to P_j on Mar 30, but the original plan P_i was still significantly faster.

RT(P_i, q_j) was 14090.69 ms, drastically outperforming the chosen RT(P_j, q_j) of 65502.06 ms.

The switch should have occurred later.

pg_hint_plan Directives Used

Switch 1

- Force Pi

```
/*+ Leading(lineitem) BitmapScan(lineitem) */
```

- Force Pj

```
/*+ Leading(lineitem) IndexScan(lineitem idx_lineitem_shipdate) */
```

Switch 2

- Force Pi

```
/*+ Leading(lineitem) IndexScan(lineitem idx_lineitem_shipdate) */
```

- Force Pj

```
/*+ Leading(lineitem) BitmapScan(lineitem) */
```

Switch 3

- Force Pi

```
/*+ Leading(lineitem) BitmapScan(lineitem) */
```

- Force Pj

```
/*+ Leading(lineitem) SeqScan(lineitem) */
```

2. Query 2 Analysis

Execution Times and Classification

Plan Switch	Dates (qi → qj)	RT(Pi, qi)	RT(Pj, qj)	RT(Pi, qj)	RT(Pj, qi)	Classification
S1,2	1992-01-01 → 1992-01-02	0.10 ms	97.66 ms	85.14 ms	0.02 ms	EARLY
S2,3	1992-05-13 → 1992-05-14	26873.38 ms	47537.95 ms	23095.95 ms	42558.67 ms	EARLY
S3,4	1997-11-25 → 1997-11-26	33402.00 ms	21634.67 ms	39843.94 ms	48838.36 ms	CORRECT

Justifications:

Switch 1 (1992-01-01 → 1992-01-02): EARLY

On Jan 2, the original plan Pi was still faster than the newly chosen plan Pj (85.14 ms < 97.66 ms)

The optimizer abandoned Pi prematurely.

Switch 2 (1992-05-13 → 1992-05-14): EARLY

On May 14, the original plan Pi was still much faster than the new plan Pj (23095.95 ms < 47537.95 ms)

The switch happened too soon.

Switch 3 (1997-11-25 → 1997-11-26): CORRECT

The optimizer switched at the correct time.

Pi was faster at qi (33402.00 ms < 48838.36 ms), and Pj was faster at qj (21634.67 ms < 39843.94 ms).

pg_hint_plan Directives Used:

Switch 1

- Force Pi

```
/*+ Leading(orders lineitem)
   IndexScan(orders idx_orders_orderdate)
   IndexScan(lineitem lineitem_pkey)
   NestLoop(orders lineitem) */
```

- Force Pj

```
/*+ Leading(orders lineitem)
   BitmapScan(orders)
   IndexScan(lineitem lineitem_pkey)
   NestLoop(orders lineitem) */
```

Switch 2

- Force Pi

```
/*+ Leading(orders lineitem)
   BitmapScan(orders)
   IndexScan(lineitem lineitem_pkey)
   NestLoop(orders lineitem) */
```

- Force Pj

```
/*+ Leading(orders lineitem)
   SeqScan(orders)
   SeqScan(lineitem)
   HashJoin(orders lineitem) */
```

Switch 3

- Force Pi

```
/*+ Leading(orders lineitem)
   SeqScan(orders)
   SeqScan(lineitem)
```

```
HashJoin(orders lineitem) */
```

- Force Pj

```
/*+ Leading(orders lineitem)
SeqScan(orders)
SeqScan(lineitem)
HashJoin(orders lineitem) */
```

Note: Plan divergence here was driven by aggregation methodology, not scan/join methods.

3. Query 3 Analysis

Execution Times and Classification

Plan Switch	Dates (qi → qj)	RT(Pi, qi)	RT(Pj, qj)	RT(Pi, qj)	RT(Pj, qi)	Classification
S1,2	1992-01-01 → 1992-01-02	0.02 ms	118.45 ms	107.78 ms	0.02 ms	EARLY
S2,3	1992-01-03 → 1992-01-04	191.38 ms	263.90 ms	244.28 ms	155.39 ms	DELAYED
S3,4	1992-01-06 → 1992-01-07	584.77 ms	673.55 ms	659.92 ms	626.39 ms	EARLY
S4,5	1992-01-18 → 1992-01-19	886.19 ms	17409.65 ms	14173.87 ms	758.34 ms	EARLY
S5,6	1992-02-14 → 1992-02-15	17920.22 ms	14256.06 ms	15829.76 ms	49741.92 ms	CORRECT
S6,7	1992-03-08 → 1992-03-09	17122.60 ms	16788.16 ms	14929.00 ms	17452.34 ms	EARLY
S7,8	1992-03-13 → 1992-03-14	22809.63 ms	19786.92 ms	141305.54 ms	17669.26 ms	DELAYED
S8,9	1992-05-16 → 1992-05-17	20829.98 ms	15894.62 ms	16344.35 ms	38033.32 ms	CORRECT

Justifications

Switch 1 (1992-01-01 → 1992-01-02): EARLY

The optimizer switched to P_j on Jan 2, but the original plan P_i was still slightly faster on that date. The forced execution $RT(P_i, q_j)$ was 107.78 ms, outperforming the optimizer's chosen $RT(P_j, q_j)$ of 118.45 ms.

Therefore, the optimizer abandoned P_i prematurely.

Switch 2 (1992-01-03 → 1992-01-04): DELAYED

The optimizer switched to P_j on Jan 4, but the new plan P_j was already outperforming P_i on Jan 3. The forced execution $RT(P_j, q_i)$ was 155.39 ms, which is faster than the optimizer's chosen $RT(P_i, q_i)$ of 191.38 ms.

The switch should have occurred sooner.

Switch 3 (1992-01-06 → 1992-01-07): EARLY

On Jan 7 (q_j), the original plan P_i was still faster than the new plan P_j . The forced execution $RT(P_i, q_j)$ of 659.92 ms beat the optimizer's chosen $RT(P_j, q_j)$ of 673.55 ms.

Therefore, the switch happened too early.

Switch 4 (1992-01-18 → 1992-01-19): EARLY

The optimizer transitioned to P_j on Jan 19, but it was a premature move. The original plan P_i was still outperforming the newly chosen plan P_j on that date i.e. $RT(P_i, q_j) = 14173.87$ ms and $RT(P_j, q_j) = 17409.65$ ms.

Therefore, the switch happened too early.

Switch 5 (1992-02-14 → 1992-02-15): CORRECT

The optimizer perfectly timed this switch. At q_i (Feb 14), P_i was the vastly superior plan (17920.22 ms < 49741.92 ms). At q_j (Feb 15), P_j proved to be the better choice over forcing P_i (14256.06 ms < 15829.76 ms).

The optimizer made the switch at the correct time.

Switch 6 (1992-03-08 → 1992-03-09): EARLY

The transition to P_j on Mar 9 happened too soon. On that day, the original plan P_i was still the faster option i.e. $RT(P_i, q_j) = 14929.00$ ms and $RT(P_j, q_j) = 16788.16$ ms.

Therefore, the switch happened too early.

Switch 7 (1992-03-13 → 1992-03-14): DELAYED

The optimizer held onto Pi for too long. By Mar 13 (qi), the new plan Pj was already vastly superior i.e $RT(Pj, qi) = 17669.26$ ms and $RT(Pi, qi) = 22809.63$ ms.

The switch should have happened earlier.

Switch 8 (1992-05-16 → 1992-05-17): CORRECT

The final switch was timed perfectly.

Pi was the correct choice on May 16 since it was faster than forcing Pj (20829.98 ms < 38033.32 ms). Conversely, Pj was the right choice on May 17, beating the forced Pi execution (15894.62 ms < 16344.35 ms).

pg_hint_plan Directives Used

Switch 1

- **Force Pi**

```
/*+ Leading(o c l)
   IndexScan(o idx_orders_orderdate)
   IndexScan(c customer_pkey)
   IndexScan(l lineitem_pkey)
   NestLoop(o c l)
   NestLoop(o c) */
```

- **Force Pj**

```
/*+ Leading(o c l)
   BitmapScan(o)
   IndexScan(c customer_pkey)
   IndexScan(l lineitem_pkey)
   NestLoop(o c l)
   NestLoop(o c) */
```

Switch 2

- **Force Pi**

```
/*+ Leading(o c l)
BitmapScan(o)
IndexScan(c customer_pkey)
IndexScan(l lineitem_pkey)
NestLoop(o c l)
NestLoop(o c) */
```

- **Force Pj**

```
/*+ Leading(c o l)
SeqScan(c)
BitmapScan(o)
IndexScan(l lineitem_pkey)
NestLoop(c o l)
HashJoin(c o) */
```

Switch 3

- **Force Pi**

```
/*+ Leading(c o l)
SeqScan(c)
BitmapScan(o)
IndexScan(l lineitem_pkey)
NestLoop(c o l)
HashJoin(c o) */
```

- **Force Pj**

```
/*+ Leading(o c l)
BitmapScan(o)
SeqScan(c)
IndexScan(l lineitem_pkey)
NestLoop(o c l)
HashJoin(o c) */
```

Switch 4

- **Force Pi**

```
/*+ Leading(o c l)
BitmapScan(o)
SeqScan(c)
IndexScan(l lineitem_pkey)
NestLoop(o c l)
HashJoin(o c) */
```

- **Force Pj**

```
/*+ Leading(l o c)
SeqScan(l)
BitmapScan(o)
SeqScan(c)
HashJoin(l o c)
HashJoin(l o) */
```

Switch 5:

- **Force Pi**

```
/*+ Leading(l o c)
SeqScan(l)
BitmapScan(o)
SeqScan(c)
HashJoin(l o c)
HashJoin(l o) */
```

- **Force Pj**

```
/*+ Leading(l o c)
SeqScan(l)
SeqScan(o)
SeqScan(c)
HashJoin(l o c)
HashJoin(l o) */
```

Switch 6

- **Force Pi**

```
/*+ Leading(l o c)
SeqScan(l)
SeqScan(o)
SeqScan(c)
HashJoin(l o c)
HashJoin(l o) */
```

- **Force Pj**

```
/*+ Leading(l o c)
SeqScan(l)
SeqScan(o)
SeqScan(c)
HashJoin(l o c)
HashJoin(o c) */
```

Switch 7

- **Force Pi**

```
/*+ Leading(l o c)
SeqScan(l)
SeqScan(o)
SeqScan(c)
HashJoin(l o c)
HashJoin(o c) */
```

- **Force Pj**

```
/*+ Leading(l o c)
SeqScan(l)
BitmapScan(o)
SeqScan(c)
HashJoin(l o c)
HashJoin(o c) */
```

Switch 8

- **Force Pi**

```
/*+ Leading(l o c)
BitmapScan(o)
SeqScan(c)
HashJoin(l o c)
HashJoin(o c) */
```

- **Force Pj**

```
/*+ Leading(l o c)
SeqScan(l)
SeqScan(o)
SeqScan(c)
HashJoin(l o c)
HashJoin(o c) */
```